

Sanity check: which NUTS polygons are outside the raster?

```
import geopandas as gpd
import rasterio
from rasterio.mask import mask
from collections import Counter

from shapely.geometry import box
from shapely.ops import unary_union
from shapely import Polygon, MultiPolygon
import rioarray
import glob

import os.path as path
import utils
import rioarray as rxr
from rioarray.merge import merge_arrays

import shapely
import xarray as xr
import numpy as np
import json
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon as MplPolygon
from concurrent.futures import ThreadPoolExecutor, as_completed

crop_type_files = sorted(glob.glob(path.join(utils.raw_data_dir, "Crop_Types", "***" , "*.tif

nuts3_boundaries_gdf = gpd.read_file(path.join(utils.raw_data_dir, "NUTS_RG_01M_2021_4326.shp
# filter only NUTS level 3
nuts3_boundaries_gdf = nuts3_boundaries_gdf[nuts3_boundaries_gdf["LEVL_CODE"] == 3]
```

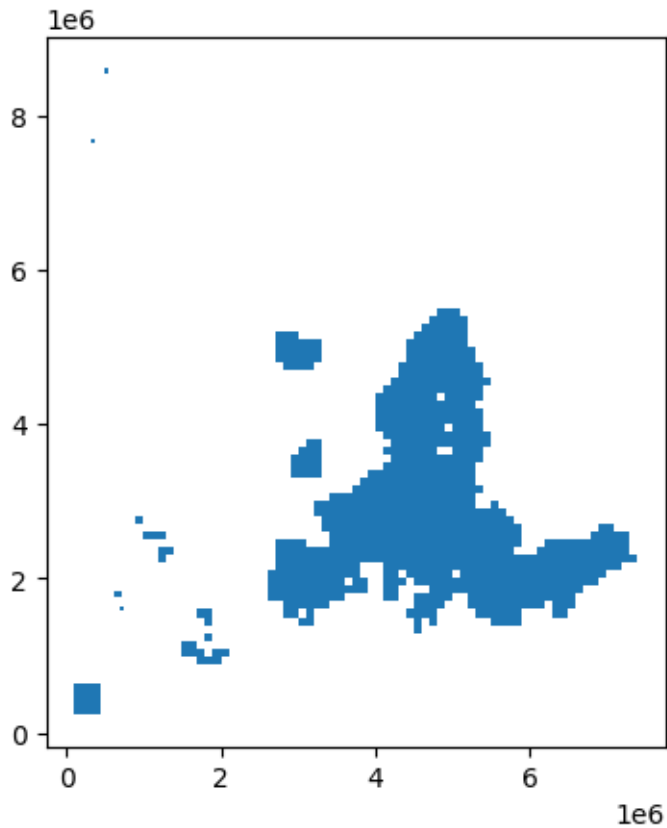
```
# reproject the nuts3-boundaries to the CRS of the cropland map. It is important to project :
# every pixel in the cropland map is exactly 10x10m, as defined by the crs.
with rasterio.open(crop_type_files[0]) as src:
    raster_crs = src.crs
nuts3_boundaries_gdf = nuts3_boundaries_gdf.to_crs(raster_crs)
assert raster_crs == nuts3_boundaries_gdf.crs
```

```
# open and index all of the tiles in the cropland dataset to allow for computationally effic
print("building tile index...")
records = []
for i, filepath in enumerate(crop_type_files):
    if i % 100 == 0:
        print(f"processing file {i} of {len(crop_type_files)}")
    with rasterio.open(filepath) as src:
        records.append({
            "path": filepath,
            "geometry": box(*src.bounds),
            "crs": src.crs,
            "bounds": src.bounds
        })

tile_index = gpd.GeoDataFrame(records, crs=records[0]["crs"])
print("Done.")
```

```
building tile index...
processing file 0 of 850
processing file 100 of 850
processing file 200 of 850
processing file 300 of 850
processing file 400 of 850
processing file 500 of 850
processing file 600 of 850
processing file 700 of 850
processing file 800 of 850
Done.
```

```
# compute a unified geometry based on all cropland tiles and plot it as a sanity check.
cropmap_boundary = shapely.ops.unary_union([box(*b) for b in tile_index["bounds"].values])
poly_is_inside_boundary = nuts3_boundaries_gdf.within(cropmap_boundary)
cropmap_boundary = gpd.GeoDataFrame(geometry=[cropmap_boundary], crs=raster_crs)
cropmap_boundary.plot()
```



- french overseas areas
- the northernmost parts of norway
- UK, which is not covered by the HRL

-> Filter is sensible and can be applied

```
nuts3_boundaries_gdf[~poly_is_inside_boundary]
```

	NUTS_ID	LEVL_CODE	CNTR_CODE	NAME_LATN	NUTS_NAME
1071	FRY30	3	FR	Guyane	Guyane
1088	FRY20	3	FR	Martinique	Martinique
1091	FRY40	3	FR	La Réunion	La Réunion
1261	FRY10	3	FR	Guadeloupe	Guadeloupe
1337	FRY50	3	FR	Mayotte	Mayotte
...	...	...	...	...	...
2001	UKM64	3	UK	Na h-Eileanan Siar (Western Isles)	Na h-Eileanan Siar
2002	UKG37	3	UK	Sandwell	Sandwell

	NUTS_ID	LEVL_CODE	CNTR_CODE	NAME_LATN	NUTS_NAME
2004	UKM77	3	UK	Perth & Kinross and Stirling	Perth & Kinross and Stirling
2005	NO0B1	3	NO	Jan Mayen	Jan Mayen
2009	NO0B2	3	NO	Svalbard	Svalbard

```
nuts3_boundaries_gdf = nuts3_boundaries_gdf[poly_is_inside_boundary]
nuts3_boundaries_gdf = nuts3_boundaries_gdf.sort_values(by="NUTS_ID")
nuts3_boundaries_gdf["crop_profile"] = None
country_list = np.unique(nuts3_boundaries_gdf["CNTR_CODE"].values)
```

```
def clip_raster(raster_path, geom_list):
    """opens a raster image and returns the value counts of all pixels that lie within a list of polygons

    Args:
        raster_path (str): path to a rasterfile that can be read by rasterio (typically geotiff)
        geom_list (list(shapely.Polygon)): list of Polygons (representating a Multipolygon)

    Returns:
        collections.Counter: value counts of all pixels in the raster image that intersect with the polygons
    """
    with rasterio.open(raster_path) as src:
        cropped_image, _ = mask(
            src,
            geom_list,
            crop=True
        )
        values, counts = np.unique(cropped_image, return_counts=True)
        return Counter(dict(zip(values.tolist(), counts.tolist())))
```

```
for cntr_code in country_list:
    if cntr_code.lower() == "uk":
        print("Skipping UK")
        continue # cropland mask does not exist for UK, any overlap is erroneous.
    out_file = path.join(utils.intermediate_data_dir, "nuts3_crop_profile", f"{cntr_code}.geotiff")
    if path.isfile(out_file):
        print(f"Output file for country {cntr_code} already exists ({out_file})")
        print("skipping...")
        continue
    print("processing country: ", cntr_code)

    temp_df = nuts3_boundaries_gdf[nuts3_boundaries_gdf["CNTR_CODE"] == cntr_code]
```

```

tile_dict = {}
tile_data = xr.DataArray()

for row_num, (row_idx, lau) in enumerate(temp_df.iterrows()):
    if row_num % 25 == 0:
        print(f"processing row {row_num} of {len(temp_df)}")
    lau_geom = lau.geometry
    matches = tile_index.sindex.query(lau_geom, predicate="intersects")
    path_list = tile_index.iloc[matches]["path"].values

    global_counter = Counter()
    if isinstance(lau_geom, Polygon):
        geom_list = [lau_geom]
    elif isinstance(lau_geom, MultiPolygon):
        geom_list = list(lau_geom.geoms)
    else:
        raise ValueError("Non-Polygon geometry: ", type(lau_geom))

    with ThreadPoolExecutor(max_workers=len(path_list)) as executor:
        futures = [executor.submit(clip_raster, rasterfile, geom_list) for rasterfile in

        for future in as_completed(futures):
            global_counter += future.result()

    temp_df.at[row_idx, "crop_profile"] = dict(global_counter)

print("writing to file: ", out_file)
temp_df.to_file(out_file)

```

Output file for country AL already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/AL/AL_2026_4_drought_data.nc) skipping...

Output file for country AT already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/AT/AT_2026_4_drought_data.nc) skipping...

Output file for country BE already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/BE/BE_2026_4_drought_data.nc) skipping...

Output file for country BG already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/BG/BG_2026_4_drought_data.nc) skipping...

Output file for country CH already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/CH/CH_2026_4_drought_data.nc) skipping...

Output file for country CY already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/CY skipping...
Output file for country CZ already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/CZ skipping...
Output file for country DE already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/DE skipping...
Output file for country DK already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/DK skipping...
Output file for country EE already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/EE skipping...
Output file for country EL already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/EL skipping...
Output file for country ES already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/ES skipping...
Output file for country FI already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/FI skipping...
Output file for country FR already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/FR skipping...
Output file for country HR already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/HR skipping...
Output file for country HU already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/HU skipping...
Output file for country IE already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/IE skipping...
Output file for country IS already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/IS skipping...
Output file for country IT already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/IT skipping...
Output file for country LI already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/LI skipping...
Output file for country LT already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/LT skipping...
Output file for country LU already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/LU skipping...
Output file for country LV already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/LV skipping...
Output file for country ME already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/ME skipping...
Output file for country MK already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/MK skipping...
Output file for country MT already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/MT skipping...
Output file for country NL already exists (/Users/johannesgille/Desktop/CE_2026_4_drought/data/NL skipping...

